JAVASCRIPT

Condicionais, repetições e coleções

Tomando decisões, repetindo tarefas e organizando dados em JavaScript.

```
> if (turma.pronta) console.log('Vamos!')
Vamos!
```

Tarik Ponciano · Senac

VISÃO GERAL

O plano de hoje

01 Abertura e retomada do conteúdo anterior

02 Condicionais: if, else e switch

Pausa

03 Repetições: for, while e do...while

Pausa

04 Coleções: arrays e objetos

05 Projeto final: CRUD do acervo da biblioteca

06 Fechamento

OBJETIVOS

Ao final da aula você vai conseguir

Decidir

com **if**, **else if**, **else** e **switch**.

Repetir

tarefas com **for**, **while** e **do...while**.

Organizar

dados em arrays e objetos.

Transformar

coleções com **map**, **filter** e **reduce**.

E juntar tudo em um CRUD completo: cadastrar, listar, editar e remover.

RETOMADA

O que já vimos antes

Sintaxe

const/**let**, tipos, **typeof**.

Exibição

console.log, template literals, formatação.

Coleta

@inquirer/prompts: **input**, **number**, **select**, **confirm**, **checkbox**.

```
const nome = await input({ message: 'Nome?' });
const idade = await number({ message: 'Idade?', required: true });
console.log(`${nome} tem ${idade} anos`);   // string, number, template literal
```

NIVELAMENTO

Quem já usou if e for em outra linguagem?

A lógica é quase sempre a mesma. Muda a sintaxe.

CONDICIONAIS

if e else

```
if (idade ⩾ 18) {
  console.log('Maior de idade');
} else {
  console.log('Menor de idade');
}
```

A condição fica entre **parênteses**.

O bloco fica entre **chaves**.

Sempre use chaves, mesmo com uma linha.

CONDICIONAIS

else if: várias faixas

```
if (nota ⩾ 9) {
  console.log('Conceito A');
} else if (nota ⩾ 7) {
  console.log('Conceito B');
} else if (nota ⩾ 5) {
  console.log('Conceito C');
} else {
  console.log('Conceito D');
}
```

Testa de cima para baixo.

Para no **primeiro** verdadeiro.

A ordem das condições importa.

CONDICIONAIS

Blocos { } criam escopo

```
if (true) {
  let mensagem = 'só existe aqui dentro';
  console.log(mensagem);
}
console.log(mensagem);   // ReferenceError: mensagem is not defined
```

{ } cria um novo **escopo**.

A variável não existe fora do bloco.

var ignora essa regra: evite.

CONDICIONAIS

if com valores truthy/falsy

```
const nome = await input({ message: 'Apelido?' });
if (nome) {
  console.log(`Oi, ${nome}!`);
} else {
  console.log('Você não digitou nada.');
}
const carrinho = [];
if (carrinho.length) {
  console.log('Tem item no carrinho');
} else {
  console.log('Carrinho vazio');
}
```

O JS converte o valor para **boolean**.

if (lista.length) = "tem itens?"

Cuidado: **0** também é falsy.

CONDICIONAIS

switch: várias opções do mesmo valor

```
switch (diaSemana) {
  case 0:
  case 6:
    console.log('Fim de semana');
    break;
  case 1: case 2: case 3: case 4: case 5:
    console.log('Dia útil');
    break;
  default:
    console.log('Dia inválido');
}
```

Compara **um valor** com várias opções.

Sem **break**, cai para o próximo (fallthrough).

default é o caso restante.

CONDICIONAIS

Relembrando o ternário

```
// já visto na aula 1: se o if/else só ESCOLHE UM VALOR, o ternário resume tudo
let status;
if (idade ⩾ 18) {
  status = 'liberado';
} else {
  status = 'bloqueado';
}

const status2 = idade ⩾ 18 ? 'liberado' : 'bloqueado';   // mesma coisa
```

Se/então/senão que **devolve um valor**.

Bom para 2 opções, uma linha.

Não aninhe ternários.

CONDICIONAIS

Erros clássicos

```
let x = 5;
if ((x = 10)) {                  // = atribui, não compara! isto sempre entra
  console.log('x agora vale', x);
}

switch (2) {
  case 1: console.log('um');
  case 2: console.log('dois');   // sem break: cai para o de baixo também
  case 3: console.log('três');
}
```

if (x = 10) atribui, não compara.

Use sempre **===**.

Sem **break**, o switch "vaza".

ATIVIDADE 1 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
let x = 7;
if (x % 2 === 0) console.log('par'); else console.log('ímpar');

let nome = '';
if (nome) console.log('tem nome'); else console.log('sem nome');

const a = 5, b = 5;
if (a === b) console.log('iguais'); else console.log('diferentes');

let n = 12;
if (n > 10 && n < 20) console.log('entre 10 e 20');
```

1. Para cada bloco, escreva o que será impresso
2. Confira com a turma (ou teste no REPL)
3. Explique o porquê de cada resposta

ATIVIDADE 2 · REPRODUZA NO COMPUTADOR E TESTE

Reescreva com switch

```
const tamanho = 2;
if (tamanho === 1) console.log('P');
else if (tamanho === 2) console.log('M');
else if (tamanho === 3) console.log('G');
else console.log('tamanho inválido');

// Reescreva com switch (tamanho) { ... }
```

1. Reproduza o código acima em um arquivo novo e rode
2. Reescreva a mesma lógica usando **switch (tamanho) { ... }**
3. Rode as duas versões com **tamanho = 1**, **2**, **3** e **9** e compare

Desafio: e se **tamanho** fosse um texto (**'P'**, **'M'**, **'G'**)? Ajuste as duas versões.

EXERCÍCIO 1 · REPRODUZA NO COMPUTADOR E TESTE

Boletim: nota → conceito

Nota	Conceito
nota ⩾ 9	A
nota ⩾ 7	B
nota ⩾ 5	C
caso contrário	D

1. Peça a nota final do aluno (**number**, de 0 a 10, com decimais)
2. Classifique com **if**/**else if**/**else**, seguindo a tabela acima
3. Mostre o resultado no formato "Nota **<nota digitada>** -> Conceito **<letra>**" (ex.: se a pessoa digitar 8.5, aparece "Nota 8.5 -> Conceito B")

Desafio: refaça a classificação usando **switch (true) { case nota >= 9: ... }**.

ATIVIDADE EM DUPLA · BALCÃO DE TRIAGEM

Central de Suporte do Senac

O Senac abriu uma central de atendimento para dúvidas técnicas (internet, impressoras, sistema acadêmico...). Vocês vão construir o sistema que direciona cada chamado para o guichê certo.

1. Pergunte: a pessoa é aluno(a) matriculado(a)? (**confirm**)
2. Pergunte a categoria do problema (**select**: Internet, Impressora, Sistema Acadêmico, Outro)
3. Pergunte a urgência (**select**: Baixa, Média, Alta)

ATIVIDADE EM DUPLA · BALCÃO DE TRIAGEM

Regras de roteamento

Decida o destino de CADA chamado testando estas 4 regras NESTA ordem — a primeira que se encaixar decide, e você para de testar as próximas:

1. **1.** Sem matrícula confirmada → sempre "Recepção — verificar cadastro" (ignore categoria e urgência)
2. **2.** Urgência Alta **e** categoria Sistema Acadêmico → "Suporte Nível 2 (urgente)"
3. **3.** Urgência Alta, qualquer OUTRA categoria → "Guichê 1 (prioritário)"
4. **4.** Nenhuma das anteriores → direcione por categoria com **switch**: Internet → Guichê 2, Impressora → Guichê 3, Sistema Acadêmico → Guichê 4, Outro → Guichê Geral
5. **5.** Em TODOS os casos (não só na regra 4), mostre também um tempo estimado de espera: Alta = 5 min, Média = 10 min, Baixa = 20 min

CHECKLIST DE ACEITE

- ☐ Regra 1 bloqueia quem não é matriculado antes de checar qualquer outra coisa
- ☐ **switch** cobre as 4 categorias da regra 4
- ☐ Regra 2 usa **&&** (urgência alta E sistema acadêmico, as duas juntas)
- ☐ Mostra o tempo de espera certo: Alta 5 min, Média 10 min, Baixa 20 min

ATIVIDADE EM DUPLA · BALCÃO DE TRIAGEM

Como o programa funciona

PASSO A PASSO

1. Faça as 3 perguntas, nesta ordem: matrícula (**confirm**), categoria e urgência (**select**)
2. Descubra o guichê: teste as regras do slide anterior de cima para baixo (**if**/**else if**)
3. Descubra o tempo de espera olhando **só** a urgência
4. Mostre a "senha": guichê + tempo de espera

EXEMPLO DE EXECUÇÃO

```
✓ A pessoa é aluno(a) matriculado(a)? Yes
✓ Categoria do problema? Internet
✓ Urgência? Alta

═══ Senha de atendimento ═══
Encaminhado para: Guichê 1 (prioritário)
Tempo estimado de espera: 5 minutos

# outro caso: pessoa sem matrícula
✓ A pessoa é aluno(a) matriculado(a)? No
✓ Categoria do problema? Impressora
✓ Urgência? Média
Encaminhado para: Recepção — verificar cadastro
Tempo estimado de espera: 10 minutos
```

INTERVALO

Pausa

Na volta: repetições (for, while)

REPETIÇÕES

for: quando se sabe quantas vezes

```
// início; condição; incremento
for (let i = 1; i ⩽ 5; i++) {
  console.log('volta', i);
}
// acumulador: somar de 1 a 100
let soma = 0;
for (let i = 1; i ⩽ 100; i++) soma += i;
console.log('Soma:', soma);       // 5050
// passo de 2: só os pares
for (let i = 2; i ⩽ 10; i += 2) console.log(i);
```

3 partes: **início; condição; incremento**.

Roda enquanto a condição for **true**.

Ideal quando já se sabe o total de voltas.

ATIVIDADE 3 · REPRODUZA NO COMPUTADOR E TESTE

Tabuada

1. Pergunte um número com **number()**
2. Use um **for** para mostrar a tabuada desse número, de 1 a 10
3. Formato: "7 x 1 = 7", "7 x 2 = 14", ...

Desafio: peça também o limite final da tabuada (em vez de fixar em 10).

REPETIÇÕES

while: enquanto for verdade

```
// em quantos anos R$ 1000 dobram, rendendo 10% ao ano?
let saldo = 1000;
let anos = 0;
while (saldo < 2000) {
  saldo = saldo * 1.1;
  anos++;
}
console.log(anos, 'anos');          // 8 anos
console.log(saldo.toFixed(2));      // 2143.59
```

Repete **enquanto** for **true**.

Sem partes fixas como o **for**.

Bom quando não se sabe o total de voltas.

ATIVIDADE 4 · REPRODUZA NO COMPUTADOR E TESTE

Contagem regressiva

1. Pergunte um número inicial com **number()**
2. Use um **while** para mostrar a contagem regressiva, um número por linha, até chegar em 1
3. Ao final, mostre "Lançamento!"

Desafio: e se o número inicial já for 0 ou negativo? Trate esse caso antes do while (nem começar a contagem).

REPETIÇÕES

do...while: roda ao menos uma vez

```
import { number } from '@inquirer/prompts';

let idade;
do {
  idade = await number({ message: 'Idade (0 a 120):', required: true });
} while (idade < 0 || idade > 120);   // testa DEPOIS de perguntar
console.log('Idade aceita:', idade);
```

Testa a condição **depois** de rodar.

Garante ao menos **uma** execução.

Bom para "peça e valide".

ATIVIDADE 5 · REPRODUZA NO COMPUTADOR E TESTE

Adivinhe até acertar

1. Sorteie um número secreto de 1 a 10: **Math.floor(Math.random() * 10) + 1**
2. Peça um palpite com **number()** — precisa perguntar **ao menos uma vez** antes de checar, por isso é um **do...while**
3. Repita pedindo o palpite enquanto ele for diferente do número secreto
4. Ao acertar, mostre "Acertou!" e quantas tentativas foram feitas

Desafio: depois de um palpite errado, dê uma dica: "maior" ou "menor".

REPETIÇÕES

break e continue

```
// break: achar o primeiro múltiplo de 7 maior que 50
for (let i = 51; i ⩽ 100; i++) {
  if (i % 7 === 0) {
    console.log('Achei:', i);   // 56
    break;                      // para o laço agora
  }
}
// continue: de 1 a 10, pulando os múltiplos de 3
for (let i = 1; i ⩽ 10; i++) {
  if (i % 3 === 0) continue;    // próxima volta
  console.log(i);
}
```

break: sai do laço **agora**.

continue: pula **esta** volta.

Funcionam em **for**, **while**, **do...while**.

REPETIÇÕES

Laço dentro de laço

```
// o laço de dentro roda INTEIRO a cada volta do de fora
for (let linha = 1; linha ⩽ 4; linha++) {
  let texto = '';
  for (let col = 1; col ⩽ linha; col++) {
    texto += '*';
  }
  console.log(texto);
}
// *
// **
// ***
// ****
```

De dentro roda **inteiro** por volta de fora.

Nomes diferentes: **linha** e **col**.

Monte a linha num **texto**.

ATIVIDADE 6 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
// bloco 1
for (let i = 0; i < 3; i++) console.log(i);
// bloco 2
let n = 5;
while (n > 0) { console.log(n); n = n - 2; }
// bloco 3
for (let i = 1; i ⩽ 5; i++) {
  if (i ═══ 3) continue;
  console.log(i);
}
```

1. Leia um bloco de cada vez, de cima para baixo
2. Para cada bloco, escreva os números que aparecem no terminal, na ordem
3. Rode os 3 blocos e compare com o que você escreveu

REPETIÇÕES

O padrão sentinela: repetir até parar

```
import { input } from '@inquirer/prompts';

let total = 0;
let quantidade = 0;
while (true) {
  const texto = await input({ message: 'Valor da venda? (Enter encerra)' });
  if (texto.trim() === '') break;   // condição de parada
  total += Number(texto.replace(',', '.'));
  quantidade++;
}
console.log(quantidade, 'vendas | total R$', total.toFixed(2));
```

while (true) roda até um **break**.

A condição de parada é um **if** por dentro.

Combina com os prompts vistos antes.

ATIVIDADE 7 · REPRODUZA NO COMPUTADOR E TESTE

Média das notas

1. Peça uma nota por vez com **input()**; apertar Enter em branco encerra a leitura
2. A cada nota, some no total e conte quantas notas foram digitadas (dica: converta com **Number(texto.replace(",", "."))**)
3. Ao encerrar, mostre a quantidade e a média com 1 casa decimal, ex.: "3 notas | média 7.5"
4. Se nenhuma nota foi digitada, mostre "Nenhuma nota informada" em vez da média

Desafio: mostre também a maior nota digitada (sem usar array: guarde numa variável).

REPETIÇÕES

Erros clássicos

```
// off-by-one: errar o ⩽ por <
for (let i = 0; i < 5; i++) {}   // 5 voltas (0 a 4)
for (let i = 0; i ⩽ 5; i++) {}  // 6 voltas (0 a 5)

// esquecer de atualizar a variável: LAÇO INFINITO
let i = 0;
while (i < 5) {
  console.log(i);
  // faltou i++ aqui!
}
```

> **<** × **<=** muda o total de voltas.

> Sem atualizar a variável: **laço infinito**.

> **Ctrl+C** interrompe o programa travado.

EXERCÍCIO 2 · REPRODUZA NO COMPUTADOR E TESTE

Repetições na prática

1. **(a) FizzBuzz:** de 1 a 30, mostre "Fizz" se o número for múltiplo de 3, "Buzz" se for de 5, "FizzBuzz" se for dos dois, ou o próprio número nos outros casos
2. **(b) Soma:** some todos os múltiplos de 3 ou de 5 de 1 a 99 e mostre o total
3. **(c) Primo:** pergunte um número com **number()** e mostre "X é primo" ou "X não é primo" (primo = só divisível por 1 e por ele mesmo)
4. **(d) Tabela:** mostre a tabela de multiplicação de 1 a 5: 5 linhas, cada uma com 5 produtos separados por espaço (dica: laço dentro de laço)

Desafio: no primo, teste só até a raiz do número e pare no primeiro divisor (break); na tabela, alinhe as colunas com **padStart(3)**.

ATIVIDADE EM DUPLA · TURNO DE CAIXA

Cantina do Senac

É hora do fechamento de caixa da cantina. Vocês vão programar o sistema que registra as vendas de um turno, uma de cada vez, até o operador encerrar.

1. Repita (padrão sentinela): pergunte o **item vendido** (**input**); Enter em branco encerra o turno
2. Para cada item, pergunte o **preço unitário** e a **quantidade** (**number**)

ATIVIDADE EM DUPLA · TURNO DE CAIXA

Regras do turno

Para cada venda registrada, calcule o valor dela (preço × quantidade) e some nos totais do turno:

1. **Desconto por venda:** se a quantidade daquela venda for 5 ou mais do MESMO item, aplique 10% de desconto só NAQUELA venda (não no turno todo)
2. Mantenha 3 acumuladores crescendo a cada venda: **total do turno** (em R$), **quantidade de vendas** (contador) e **quantas vendas tiveram desconto** (contador)
3. Quando o operador encerrar (Enter em branco no nome do item), mostre o resumo com os 3 acumuladores
4. **Meta do turno:** se o total acumulado for R$ 100 ou mais, mostre uma mensagem de "meta batida"; se for menor, mostre quanto falta (100 – total)

CHECKLIST DE ACEITE

- ☐ Usa o padrão sentinela (**while (true)** + **break** no nome em branco) para repetir as vendas
- ☐ Os 3 acumuladores (total, quantidade de vendas, vendas com desconto) fecham certos no resumo
- ☐ O desconto de 10% olha a quantidade de CADA venda, não o total acumulado
- ☐ A mensagem de meta bate com o valor: "batida" ou "faltam R$ X", nunca os dois juntos

ATIVIDADE EM DUPLA · TURNO DE CAIXA

Como o programa funciona

PASSO A PASSO

1. Pergunte o item; se vier em branco, saia do laço (**break**)
2. Pergunte preço e quantidade e calcule preço × quantidade
3. Se a quantidade for 5 ou mais, tire 10% **desta** venda
4. Mostre o valor, some nos 3 acumuladores e volte ao passo 1
5. Depois do laço: mostre o fechamento e a meta

EXEMPLO DE EXECUÇÃO

```
✓ Item vendido? (Enter encerra) Coxinha
✓ Preço unitário (R$)? 6.5
✓ Quantidade? 2
  Coxinha: R$ 13.00
✓ Item vendido? (Enter encerra) Suco
✓ Preço unitário (R$)? 5
✓ Quantidade? 6
  → desconto de 10% aplicado
  Suco: R$ 27.00
✓ Item vendido? (Enter encerra)
═══ Fechamento do caixa ═══
Vendas realizadas: 2
Vendas com desconto: 1
Total do turno: R$ 40.00
Faltaram R$ 60.00 para bater a meta.
```

INTERVALO

Pausa

Na volta: coleções (arrays e objetos)

COLEÇÕES · ARRAYS

Criar e acessar

```
const numeros = [5, 3, 8, 1, 9];
console.log(numeros[0], numeros.length);   // 5 5
console.log(numeros.at(-1));               // 9 (último)
numeros[1] = 30;                           // altera pelo índice
console.log(numeros);
```

Índices começam em **0**.

length conta os itens.

at(-1) pega o último.

COLEÇÕES · ARRAYS

Adicionar e remover

```
const fila = ['Ana'];
fila.push('Bia');        // adiciona no FIM
fila.unshift('Caio');    // adiciona no INÍCIO
console.log(fila);       // ['Caio', 'Ana', 'Bia']

fila.pop();              // remove do FIM (e devolve o valor)
fila.shift();            // remove do INÍCIO
console.log(fila);       // ['Ana']
```

push/**pop**: no **fim**.

unshift/**shift**: no **início**.

Todos alteram o array original.

COLEÇÕES · ARRAYS

Percorrer: for × for...of

```
const presentes = ['Ana', 'Bia', 'Caio'];

// for clássico: usa o ÍNDICE (a posição)
for (let i = 0; i < presentes.length; i++) {
  console.log(i + 1, presentes[i]);     // 1 Ana / 2 Bia / 3 Caio
}

// for...of: pega o VALOR direto
for (const nome of presentes) {
  console.log('Olá,', nome);            // Olá, Ana / Olá, Bia / ...
}
```

for: dá o **índice** (posição).

for...of: dá o **valor** direto.

Os dois aceitam **break** e **continue**.

COLEÇÕES · ARRAYS

forEach: uma ação para cada item

```
const compras = ['pão', 'leite', 'café'];

// forEach chama a função UMA VEZ para cada item
compras.forEach((item) ⇒ {
  console.log('Comprar:', item);
});

// o 2º parâmetro é o índice
compras.forEach((item, i) ⇒ {
  console.log(`${i + 1}. ${item}`);     // 1. pão / 2. leite / 3. café
});
```

Recebe uma **função**, chamada 1x por item.

2º parâmetro: o **índice**.

Não devolve nada e **não para** com **break**.

ATIVIDADE 8 · REPRODUZA NO COMPUTADOR E TESTE

Lista de presença

```
const presentes = ['Ana', 'Bia', 'Caio', 'Davi'];
```

1. Copie o array **presentes** do painel acima
2. Com **forEach**, mostre cada nome numerado, um por linha: "1. Ana", "2. Bia", ...
3. Depois do **forEach**, mostre o total: "Total: 4 presentes" (dica: **presentes.length**)

Desafio: antes de mostrar, acrescente mais um nome com **push** e veja se a numeração e o total se ajustam sozinhos.

COLEÇÕES · ARRAYS

Buscar dentro do array

```
const numeros = [5, 30, 8, 1, 9];
numeros.includes(8);                    // true
numeros.indexOf(8);                     // 2
numeros.find((n) ⇒ n > 20);             // 30 (primeiro que passa)
numeros.some((n) ⇒ n > 20);             // true (existe algum?)
numeros.every((n) ⇒ n > 0);             // true (todos passam?)
```

includes/**indexOf**: valor exato.

find: primeiro que passa no teste.

some/**every**: existe algum? / todos?

COLEÇÕES · ARRAYS

map e filter

```
const numeros = [3, 7, 1, 9];

numeros.map((n) ⇒ n + 1);        // [4, 8, 2, 10] — transforma cada item
numeros.filter((n) ⇒ n > 5);     // [7, 9]        — mantém quem passa no teste
// os dois SEMPRE devolvem um array NOVO; o original não muda
```

map: transforma (mesmo tamanho).

filter: seleciona (tamanho pode mudar).

Sempre devolvem um array **novo**.

COLEÇÕES · ARRAYS

reduce: junta tudo em um valor

```
[10, 20, 30].reduce((acumulado, atual) ⇒ acumulado + atual, 0);

// passo a passo:
// acumulado=0  , atual=10  →  devolve 10
// acumulado=10 , atual=20  →  devolve 30
// acumulado=30 , atual=30  →  devolve 60
// resultado final: 60
```

Acumula um resultado a cada volta.

O 2º argumento é o **valor inicial**.

Bom para somas, totais e contagens.

COLEÇÕES · ARRAYS

Qual usar para percorrer?

Forma	Use quando...	break?
for	precisa do índice ou de outro caminho (de trás para frente, de 2 em 2)	sim
for...of	quer cada valor, um por vez: o jeito padrão de percorrer	sim
forEach	quer executar uma ação para cada item (mostrar, somar numa variável)	não
map/**filter**/**reduce**	quer gerar um array novo ou um único valor	não

Com **await** (prompts) dentro da repetição, use **for** ou **for...of**: o **forEach** não espera a resposta.

COLEÇÕES · ARRAYS

sort: cuidado com a pegadinha

```
[10, 9, 1].sort();                    // [1, 10, 9]   compara como TEXTO!
[10, 9, 1].sort((a, b) ⇒ a - b);      // [1, 9, 10]   comparador numérico
// sort() ALTERA o array original!
// copie antes: [...arr].sort(...)
```

Sem comparador, ordena como **texto**.

Números: **(a, b) => a - b**.

sort altera o array original.

COLEÇÕES · ARRAYS

Desestruturação e espalhamento

```
const [primeiro, segundo, ...resto] = [1, 2, 3, 4, 5];
console.log(primeiro, segundo, resto);   // 1 2 [3, 4, 5]

const copia = [...resto, 999];           // copia + acrescenta, sem alterar "resto"
console.log(copia, resto);
```

const [a, b, ...resto] = arr

[...arr, novoItem] copia + acrescenta

Nunca altera o array original.

ATIVIDADE 9 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
const n = [3, 7, 1, 9];
console.log(n.map((x) ⇒ x * 2));            // A
console.log(n.filter((x) ⇒ x > 5));         // B
console.log(n.reduce((s, x) ⇒ s + x, 0));   // C
```

1. Para cada linha marcada (A, B e C), escreva o que aparece no terminal
2. Lembre: **map** devolve um array do **mesmo tamanho**; **filter**, só os itens que passam no teste; **reduce**, **um único valor**
3. Rode o código e confira

ATIVIDADE 10 · REPRODUZA NO COMPUTADOR E TESTE

Filtre e some

```
const precos = [45, 120, 30, 89, 15];
// filtre os preços acima de 50 e depois some o total com reduce
```

1. Copie o array **precos** do painel acima em um arquivo novo
2. Com **filter**, gere um novo array só com os preços acima de 50
3. Com **reduce**, some os valores desse novo array e mostre o total

Desafio: faça tudo em uma linha só, encadeando **.filter(...).reduce(...)**.

EXERCÍCIO 3 · REPRODUZA NO COMPUTADOR E TESTE

Notas da turma

```
const notas = [7.5, 4.0, 9.0, 6.5, 10, 3.5, 8.0];
```

1. Copie o array **notas** do painel acima
2. Mostre cada nota com **forEach**, no formato "Aluno 1: 7.5"
3. Calcule a **média**: some as notas com **reduce**, divida por **notas.length** e mostre com 1 casa decimal
4. Conte os **aprovados** (nota 7 ou mais) com **filter** e mostre "Aprovados: 4 de 7"

Desafio: mostre a maior e a menor nota (**Math.max(...notas)** e **Math.min(...notas)**) e o percentual de aprovação.

COLEÇÕES · OBJETOS

Criar um objeto

```
const aluno = {
  nome: 'Ana',
  idade: 20,
  notas: [8, 9, 7],
};
console.log(aluno.nome, aluno.notas.length);
```

Pares **chave: valor**, entre **{ }**.

Cada valor pode ser de qualquer tipo.

Use quando os dados têm papéis diferentes.

COLEÇÕES · OBJETOS

Acessar, alterar, remover

```
console.log(aluno.nome);          // ponto
console.log(aluno['idade']);      // colchetes (para chave dinâmica)

aluno.idade = 21;                 // altera
aluno.email = 'ana@ex.com';       // cria, se não existia
delete aluno.email;               // remove
```

Ponto ou **colchetes** para acessar.

Atribuir numa chave nova **cria** a propriedade.

delete remove uma propriedade.

COLEÇÕES · OBJETOS

Percorrer um objeto

```
const curso = { nome: 'DSM', cargaHoraria: 1200, vagas: 30 };

Object.keys(curso);       // ['nome', 'cargaHoraria', 'vagas']
Object.values(curso);     // ['DSM', 1200, 30]
Object.entries(curso);    // [['nome','DSM'], ['cargaHoraria',1200], ['vagas',30]]

for (const [chave, valor] of Object.entries(curso)) {
  console.log(chave, '→', valor);
}
```

Object.keys/values/entries

entries devolve pares **[chave, valor]**.

Combine com **for...of**.

COLEÇÕES · OBJETOS

Objeto na prática: contando votos

```
const votos = { python: 12, javascript: 20, java: 8 };
votos.java++;                          // alterar um valor
votos.go = 3;                          // criar uma chave nova

// objeto NÃO tem forEach: passe por Object.entries antes
Object.entries(votos).forEach(([linguagem, total]) ⇒ {
  console.log(`${linguagem}: ${total} votos`);
});

let soma = 0;
Object.values(votos).forEach((n) ⇒ { soma += n; });
console.log('Total:', soma);       // 44
```

Objeto como **contador**: chave → total.

Objeto **não tem forEach**.

Object.entries(obj).forEach(...)

COLEÇÕES · OBJETOS

for...in: outra forma de percorrer chaves

```
const aluno = { nome: 'Ana', idade: 20 };
for (const chave in aluno) {
  console.log(chave, '→', aluno[chave]);
}
// for...in é para as CHAVES de um objeto. Em array, evite: dá o índice como texto.
```

Percorre as **chaves** de um objeto.

Evite em arrays: prefira **for...of**.

Hoje preferimos **Object.entries** + **for...of**.

COLEÇÕES · OBJETOS

Desestruturação e espalhamento

```
const { nome, ...resto } = aluno;
console.log(nome, resto);                // Ana { idade: 21, notas: [...] }

const { idade: idadeDoAluno = 0 } = aluno;   // renomeando + valor padrão
console.log(idadeDoAluno);

const atualizado = { ...aluno, idade: 22 };  // cópia com alteração
console.log(atualizado.idade, aluno.idade);  // 22 21
```

const { nome } = objeto

Renomear: **{ idade: idadeDoAluno }**

{ ...objeto, campo: novoValor }
copia + altera

COLEÇÕES

O padrão mais usado: array de objetos

```
const produtos = [
  { nome: 'Caderno', preco: 18.9, categoria: 'papelaria' },
  { nome: 'Mochila', preco: 129.9, categoria: 'acessórios' },
];

for (const { nome, preco } of produtos) {
  console.log(`${nome}: R$ ${preco.toFixed(2)}`);
}

produtos.filter((p) ⇒ p.categoria ═══ 'papelaria');
produtos.map((p) ⇒ p.nome.toUpperCase());
produtos.reduce((total, p) ⇒ total + p.preco, 0);
```

É como uma **tabela** guardada na memória: cada objeto é uma linha, cada propriedade é uma coluna.

COLEÇÕES · OBJETOS

Percorrer um array de objetos

```
const turma = [
  { nome: 'Ana', nota: 8.5 },
  { nome: 'Bia', nota: 6 },
  { nome: 'Caio', nota: 9.5 },
];
// for...of: cada volta recebe o objeto inteiro
for (const aluno of turma) console.log(aluno.nome, aluno.nota);

// forEach: desestruturando no parâmetro, com o índice
turma.forEach(({ nome, nota }, i) ⇒ {
  console.log(`${i + 1}. ${nome}: ${nota ⩾ 7 ? 'aprovado' : 'recuperação'}`);
});
```

Cada volta recebe **um objeto**.

Desestruture: **({ nome, nota })**.

for...of ou **forEach**: mesmo resultado.

COLEÇÕES NA PRÁTICA

Juntando tudo: carrinho de compras

```
const carrinho = [];
carrinho.push({ item: 'Caderno', preco: 18.9, qtd: 2 });
carrinho.push({ item: 'Caneta', preco: 3.5, qtd: 10 });
carrinho.push({ item: 'Mochila', preco: 129.9, qtd: 1 });

console.log('--- RECIBO ---');
carrinho.forEach(({ item, preco, qtd }, i) ⇒ {
  console.log(`${i + 1}. ${item.padEnd(8)} ${qtd} x R$ ${preco.toFixed(2)}`);
});
const total = carrinho.reduce((soma, p) ⇒ soma + p.preco * p.qtd, 0);
console.log('Total: R$', total.toFixed(2));   // 202.70
```

push monta a lista.

forEach imprime o recibo.

reduce calcula o total.

COLEÇÕES · OBJETOS

JSON: texto ↔ objeto

```
const texto = JSON.stringify(produtos);    // array de objetos → texto
console.log(texto);

const devolta = JSON.parse(texto);         // texto → array de objetos de novo
console.log(devolta[0].nome);
```

stringify: objeto → texto.

parse: texto → objeto.

É assim que dados viajam entre sistemas.

ATIVIDADE 11 · REPRODUZA NO COMPUTADOR E TESTE

Objeto e Object.entries

1. Crie um objeto **curso** com três propriedades: **nome**, **cargaHoraria** e **vagas**
2. Mostre cada propriedade no formato "chave: valor", uma por linha (dica: **Object.entries** + **for...of** ou **forEach**)

Desafio: crie uma CÓPIA do objeto com **vagas** reduzida em 1, sem alterar o **curso** original.

ATIVIDADE 12 · REPRODUZA NO COMPUTADOR E TESTE

Cardápio da cantina

```
const cardapio = [
  { nome: 'Coxinha', preco: 6.5 },
  { nome: 'Suco', preco: 5 },
  { nome: 'Pão de queijo', preco: 4 },
];
```

1. Copie o array **cardapio** do painel acima
2. Com **forEach**, mostre um item por linha: "Coxinha — R$ 6.50" (dica: **preco.toFixed(2)**)
3. Com **reduce**, mostre quanto custa pedir um de cada item

Desafio: use **map** para criar um cardápio novo com 10% de desconto e mostre-o com **forEach**.

EXERCÍCIO 4 · REPRODUZA NO COMPUTADOR E TESTE

Catálogo de produtos

```
const produtos = [
  { nome: 'Caderno', preco: 18.9, categoria: 'papelaria', estoque: 40 },
  { nome: 'Mochila', preco: 129.9, categoria: 'acessórios', estoque: 8 },
  { nome: 'Caneta', preco: 3.5, categoria: 'papelaria', estoque: 120 },
  { nome: 'Fone de ouvido', preco: 79.9, categoria: 'eletrônicos', estoque: 15 },
  { nome: 'Garrafa', preco: 45.0, categoria: 'acessórios', estoque: 3 },
];
```

1. Copie o array **produtos** do painel acima
2. Mostre o catálogo com **forEach**, um produto por linha: "Caderno — R$ 18.90"
3. Com **filter**, mostre só os produtos com preço acima de R$ 40
4. Com **reduce**, calcule o valor total do estoque: some **preco * estoque** de cada produto

Desafio: conte quantos produtos há em cada categoria (objeto contador) e mostre o produto mais barato.

ATIVIDADE EM DUPLA · ESTOQUE

Papelaria do Senac

A papelaria precisa de um sistema simples de controle de estoque. Vocês vão cadastrar os produtos e gerar um relatório da situação atual.

1. Repita (padrão sentinela): pergunte **nome**, **preço** e **quantidade** de cada produto; nome em branco encerra o cadastro
2. Guarde cada produto como um **objeto** dentro de um **array** (**estoque.push({ nome, preco, quantidade })**)

ATIVIDADE EM DUPLA · ESTOQUE

O relatório final

Depois que o cadastro terminar, mostre 3 partes, nesta ordem:

1. **Situação de cada produto:** percorra o array e classifique com **if/else if**: quantidade 0 → **ESGOTADO**; menos de 5 → **BAIXO**; senão → **OK**
2. **Itens para repor:** com **filter**, pegue os produtos com quantidade menor que 5 e mostre o nome de cada um
3. **Valor total do estoque:** com **reduce**, some preço × quantidade de todos os produtos

Desafio (opcional): mostre o produto mais caro e a lista do mais barato ao mais caro (**sort** em uma cópia: **[...estoque]**).

CHECKLIST DE ACEITE

- ☐ O cadastro usa **while (true)** + **break** (nome em branco)
- ☐ A situação usa **if/else if** com as 3 faixas
- ☐ Cada produto vira um objeto **{ nome, preco, quantidade }**
- ☐ **filter** e **reduce** batem com o exemplo de execução

ATIVIDADE EM DUPLA · ESTOQUE

Como o programa funciona

PASSO A PASSO

1. Repita: pergunte nome, preço e quantidade; nome em branco encerra
2. Guarde cada produto no array: **estoque.push({ nome, preco, quantidade })**
3. Depois do laço, percorra o **estoque** e mostre cada produto com a situação
4. Mostre os itens para repor (**filter**) e o valor total (**reduce**)

EXEMPLO DE EXECUÇÃO

```
✓ Nome do produto? (Enter encerra) Caderno
✓ Preço (R$)? 18.9
✓ Quantidade em estoque? 40
✓ Nome do produto? (Enter encerra) Mochila
✓ Preço (R$)? 129.9
✓ Quantidade em estoque? 3
✓ Nome do produto? (Enter encerra)
═══ Situação do estoque ═══
Caderno    qtd: 40   [OK]
Mochila    qtd: 3    [BAIXO]
═══ Itens para repor ═══
- Mochila (restam 3)

Valor total do estoque: R$ 1145.70
```

ATIVIDADE EM DUPLA · PROJETO FINAL

CRUD do Acervo da Biblioteca

A biblioteca do Senac quer controlar os livros pelo terminal. Vocês vão construir um **CRUD**: um programa que **cadastra** (Create), **lista** (Read), **edita** (Update) e **remove** (Delete) livros.

1. Cada livro é um objeto **{ id, titulo, autor, ano }**, guardado no array **livros**
2. O **id** é automático: o 1º livro recebe 1, o 2º recebe 2, e assim por diante (nunca se repete)
3. Um menu com **select** se repete até a pessoa escolher **Sair**. Opções: Cadastrar, Listar, Editar, Remover, Sair

ATIVIDADE EM DUPLA · PROJETO FINAL

O que cada opção do menu faz

1. **Cadastrar:** pergunte título, autor e ano; adicione o livro com **push** e mostre "Livro #1 cadastrado."
2. **Listar:** se não houver livros, mostre "Nenhum livro cadastrado."; senão, um livro por linha com **forEach**
3. **Editar:** peça o id e procure o livro com **find**; se não achar, avise; se achar, pergunte o novo título e o novo autor
4. **Remover:** peça o id e procure com **find**; se achar, peça confirmação (**confirm**) e remova com **filter**

CHECKLIST DE ACEITE

- ☐ O menu volta depois de cada opção, até escolher Sair
- ☐ Editar e Remover avisam quando o id não existe
- ☐ Os ids não se repetem, mesmo depois de remover um livro
- ☐ Listar mostra a lista já atualizada após editar/remover

ATIVIDADE EM DUPLA · PROJETO FINAL

Como o programa funciona

PASSO A PASSO

1. Mostre o menu com **select**, dentro de um **while (true)**
2. Se a opção for Sair: **break**
3. Senão, execute a opção escolhida (**if**/**else if**)
4. Terminada a opção, o laço volta ao menu (passo 1)

EXEMPLO DE EXECUÇÃO

```
✓ O que deseja fazer? Cadastrar
✓ Título: Dom Casmurro
✓ Autor: Machado de Assis
✓ Ano de publicação: 1899
Livro #1 cadastrado.

✓ O que deseja fazer? Listar
#1  Dom Casmurro — Machado de Assis (1899)
Total: 1 livro(s)

✓ O que deseja fazer? Remover
✓ Id do livro a remover: 1
✓ Remover "Dom Casmurro"? Yes
Livro #1 removido.

✓ O que deseja fazer? Sair
```

PROJETO FINAL

Esqueleto: copie e complete os 4 TODOs

```
import { input, number, select, confirm } from '@inquirer/prompts';
let livros = [];        // cada livro: { id, titulo, autor, ano }
let proximoId = 1;
while (true) {
  const opcao = await select({ message: 'O que deseja fazer?',
    choices: ['Cadastrar', 'Listar', 'Editar', 'Remover', 'Sair'] });
  if (opcao === 'Sair') break;

  if (opcao === 'Cadastrar') {
    // TODO 1: perguntar título, autor e ano; push no array; proximoId++
  } else if (opcao === 'Listar') {
    // TODO 2: vazio? avisar. Senão: forEach, um livro por linha
  } else if (opcao === 'Editar') {
    // TODO 3: pedir id; find; se achou, perguntar novo título e autor
  } else if (opcao === 'Remover') {
    // TODO 4: pedir id; find; confirm; livros = livros.filter(...)
  }
}
```

FECHAMENTO

Resumo da aula

- **Condicionais:** if, else if, else, switch, blocos e escopo
- **Repetições:** for, while, do...while, break/continue, o padrão sentinela
- **Arrays:** push/pop, for...of, forEach, map, filter, reduce, sort
- **Objetos:** propriedades, Object.entries, array de objetos
- **Tudo junto:** um CRUD completo (acervo da biblioteca)

FECHAMENTO

Próximos passos

Funções

Declarar, parâmetros, retorno, reaproveitar código.

Organização

Dividir um programa grande em partes pequenas.

Ainda mais coleções

Arrays de arrays, objetos aninhados.

Para casa: acrescente ao CRUD da biblioteca uma opção "Buscar por autor" (com **filter**) e um campo **disponivel** para emprestar e devolver livros.

FECHAMENTO

Para praticar em casa

Lista de questões

Questões sobre condicionais, laços e coleções, com respostas comentadas.

Gabarito comentado

Atividades, exercícios e as 4 atividades temáticas, resolvidos e explicados.

Como estudar

Tente antes, rode no Node e só então compare com o gabarito.

No fim da aula você recebe a **lista-de-questoes-aula2.md** e a pasta **gabarito/**.

OBRIGADO

Dúvidas e onde estudar mais

> **Recomendado: W3Schools**
>
> Páginas de if/else, switch, laços e arrays/objetos, com exemplos que você pode editar e executar. Conteúdo em inglês.
>
> Tutorial de JavaScript · Referência · Node.js

MDN Web Docs (pt-BR): developer.mozilla.org/pt-BR

javascript.info: javascript.info

Node.js: nodejs.org/docs/latest/api

@inquirer/prompts: github.com/SBoudrias/Inquirer.js